

SE446: Big Data Engineering

Week 11: Spark Structured Streaming — Real-Time Data Processing

Prof. Anis Koubaa

SE 446

Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Today's Agenda

- 1 The Streaming Mental Model
- 2 Reading Streams
- 3 Transformations on Streams
- 4 Writing Stream Results
- 5 End-to-End Pipeline
- 6 Summary

Outline

- 1 The Streaming Mental Model
- 2 Reading Streams
- 3 Transformations on Streams
- 4 Writing Stream Results
- 5 End-to-End Pipeline
- 6 Summary

Learning Objectives

By the end of this session you will be able to:

- 1 Explain the “infinite table” model of Structured Streaming
- 2 Distinguish between micro-batch and continuous processing
- 3 Read from Kafka in Spark using `readStream`
- 4 Apply `DataFrame` transformations on streaming data
- 5 Write results with `writeStream` using append, update, and complete modes
- 6 Implement tumbling and sliding time windows

The Big Idea

You already know the `DataFrame` API from Weeks 7–8. **Structured Streaming uses the exact same API** — the only difference is the input keeps growing.

The Big Data Journey — Where Are We?



Week 10 Recap

Kafka = distributed message bus.
Producers → Topics/Partitions → Consumers.

Week 11 Goal

Connect Spark to Kafka and process streams in **real time**.

Why Spark Streaming? — The Python Consumer Hits a Wall

Last week you wrote a Python Kafka consumer. It works. But can it handle production?

✗ No Parallelism

Your topic has
3 partitions.
Your Python script uses
1 thread, reads
1 partition.

2/3 of data ignored.

⚠ No Fault Tolerance

Script crashes at 3 AM.
No checkpoint saved.
On restart:
latest only.

4 hours of data lost.

🗑 No State Management

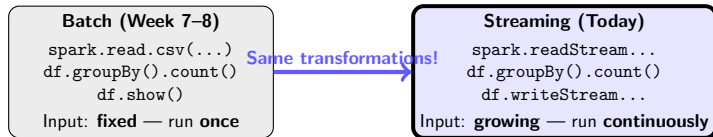
You want count
by district.
You build a Python
dict in RAM.
Script dies →
dict is gone.

All counts lost.

The Core Need

We need a framework that reads **all partitions in parallel**, **checkpoints** progress automatically, and **manages aggregation state** across restarts. That framework is **Spark Structured Streaming**.

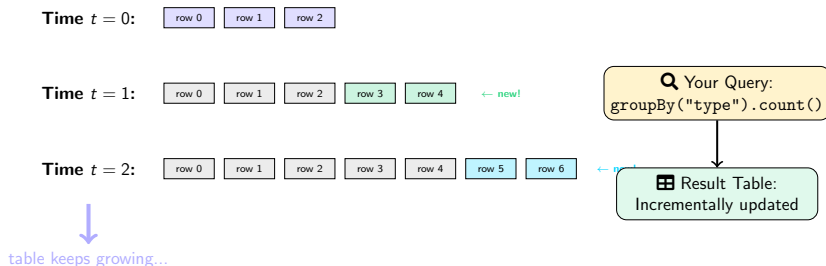
From Batch to Streaming — Same API!



💡 The Key Insight

You do **not** need to learn a new API for streaming. `filter()`, `groupBy()`, `join()` — everything works the same. Only the input source and output sink change. Every skill you built in Weeks 7–8 transfers directly to real-time processing — **no rewriting code**.

The Infinite Table Analogy

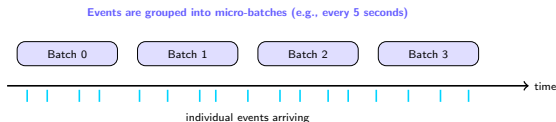


💡 Why This Abstraction Is Powerful

The “infinite table” lets you think in **SQL**, not in callbacks or event loops. Instead of writing “when a message arrives, look up the counter, increment it, handle crashes, manage memory...,” you write: `groupBy("type").count()`. Spark handles the incremental update, state management, and fault tolerance **for you**.

Micro-Batch vs Continuous Processing

Mode	How It Works	Latency
Micro-batch (default)	Accumulates data, processes as a small Spark batch	100ms – few seconds
Continuous (experimental)	Event-by-event processing, limited operations	~1ms

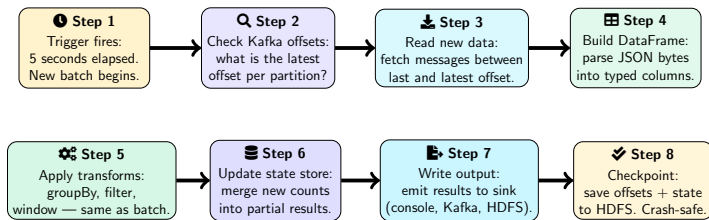


? Why Did Spark Choose Micro-Batch?

Micro-batch reuses the **entire Spark SQL engine** you learned in Week 7 — the Catalyst optimizer, the code generator, the shuffle infrastructure. Building a new event-by-event engine would have meant rewriting everything. The trade-off: **100ms latency instead of 1ms**. For 95% of use cases, that is fast enough. For sub-millisecond latency, look at Apache Flink.

Follow One Micro-Batch — Under the Hood

When a micro-batch triggers, what actually happens inside Spark? Let us trace it step by step.



Key Observations

Each micro-batch is a **complete mini Spark job**: read, transform, write. The state store and checkpoint ensure that if Spark crashes between steps 6 and 8, the batch is **re-executed from step 3** on restart — nothing is lost or double-counted.

Outline

- 1 The Streaming Mental Model
- 2 Reading Streams
- 3 Transformations on Streams
- 4 Writing Stream Results
- 5 End-to-End Pipeline
- 6 Summary

readStream — The Streaming Entry Point

```
# BATCH read (familiar from Weeks 7-8):
df = spark.read.csv("hdfs:///data/crimes.csv", header=True)

# STREAMING read from Kafka (new!):
stream_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "crimes") \
    .option("startingOffsets", "latest") \
    .load()
```

spark.read (Batch)

Returns a **static** DataFrame
(finite, all data exists now)

spark.readStream (Streaming)

Returns a **streaming** DataFrame
(infinite, new rows keep arriving)

Kafka Message Schema

The raw DataFrame has columns: key, value (both bytes), topic, partition, offset, timestamp.
You must **parse** the value column yourself.

🔗 Connection to Week 10: Partitions = Tasks

Spark creates **one task per Kafka partition**. If crimes has 3 partitions, Spark reads them with 3 parallel tasks — exactly like the consumer group you built last week, but with automatic offset tracking and fault tolerance.

Parsing Kafka Messages

Kafka stores everything as bytes. You must define a schema and parse:

```
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StringType, IntegerType, BooleanType

# Define the expected JSON schema
crime_schema = StructType() \
    .add("id", IntegerType()) \
    .add("type", StringType()) \
    .add("district", IntegerType()) \
    .add("hour", IntegerType()) \
    .add("arrest", BooleanType())

# Parse: bytes -> string -> JSON -> DataFrame columns
crimes_df = stream_df \
    .selectExpr("CAST(value AS STRING) as json_str") \
    .select(from_json(col("json_str"), crime_schema).alias("data")) \
    .select("data.*")

# Now crimes_df has columns: id, type, district, hour, arrest
```

Three-Step Pattern

(1) Cast bytes to string → (2) Parse JSON with schema → (3) Flatten nested struct into columns.

Recall from Week 10

In your Python consumer, you called `json.loads(msg.value().decode("utf-8"))`. This is the **same deserialization step** — Kafka stores bytes, and the consumer (whether Python or Spark) must parse them. The difference: Spark does it **in parallel across all partitions**.

Live Cluster Result: Schema + Parsed Events

Actual output from our SE446 cluster (Spark 3.5.4, Kafka 3.9.0, 134.209.172.50).

Raw Kafka Schema

```
root
|-- key: binary
|-- value: binary
|-- topic: string
|-- partition: integer
|-- offset: long
|-- timestamp: timestamp
|-- timestampType: integer
```

Key Observation

key and value are **binary**. You must cast and parse them yourself.

After Parsing: Structured Columns

id	type	district	hour	arrest	partition	offset
7	ASSAULT	8	11	false	2	0
10	BURGLARY	1	5	true	2	1
17	ASSAULT	1	7	false	2	2
18	BATTERY	1	8	true	2	3
1	BATTERY	4	17	true	1	0
2	ASSAULT	3	6	true	1	1
6	ASSAULT	6	22	true	1	2
8	THEFT	5	4	false	1	3
12	CRIMINAL_DAMAGE	5	13	false	1	4

Notice the Partition Column

Events are **not in id order** — they are grouped by **Kafka partition**. District 8 and 1 land on partition 2; district 4, 3, 6, 5 land on partition 1. This is the hash-partitioning from Week 10 in action:
`hash(district) % 3`.

Streaming Sources Comparison

Source	Format	Use Case	Fault-Tolerant?
kafka	Messages from topics	Production streaming	✓ Yes
socket	Text from TCP	Testing only	✗ No
rate	Generated rows/sec	Benchmarking	✓ Yes
file	New files in dir	Semi-batch (watch folder)	✓ Yes

For This Course

We use **Kafka** as our streaming source. This is the standard production pattern: Kafka → Spark Structured Streaming → output.

Recall from Week 10

Our Kafka broker runs at MASTER_IP:9092. We created topics and Python producers/consumers. Now Spark replaces the Python consumer with a much more powerful engine.

Outline

- 1 The Streaming Mental Model
- 2 Reading Streams
- 3 Transformations on Streams**
- 4 Writing Stream Results
- 5 End-to-End Pipeline
- 6 Summary

Familiar Operations on Streaming Data

```
# Filter: only thefts (same as batch!)
thefts = crimes_df.filter(col("type") == "THEFT")

# Aggregation: count by crime type
type_counts = crimes_df.groupBy("type").count()

# Aggregation: count by district
district_counts = crimes_df.groupBy("district").count()

# Multiple aggregations
stats = crimes_df.groupBy("type").agg(
    count("*").alias("total"),
    avg("hour").alias("avg_hour")
)
```

💡 The Entire Point of Structured Streaming

This code is **identical** to batch Spark SQL. You already know how to do this from Week 7. The engine handles the “streaming” part automatically.

Live Cluster Result: Streaming Aggregations

30 crime events produced to Kafka, consumed by Spark Structured Streaming. Two aggregations ran on the same stream — identical syntax to batch.

```
groupBy("type").count()
```

type	count
ASSAULT	7
BURGLARY	7
THEFT	5
CRIMINAL_DAMAGE	4
BATTERY	2
ROBBERY	2
MOTOR_VEHICLE_THEFT	2
NARCOTICS	1

```
groupBy("district").agg(count, sum)
```

district	total_crimes	arrests
1	4	3
2	3	1
3	2	1
4	3	1
5	3	0
6	1	1
7	3	0
8	2	1
9	3	2
10	1	1
11	2	1
12	3	1

This Is Real-Time

These results update **automatically** as new events arrive. If the producer sends 30 more events, the counts increment — no code changes needed. Output mode complete rewrites the full table each micro-batch.

What You CAN and CANNOT Do on Streams

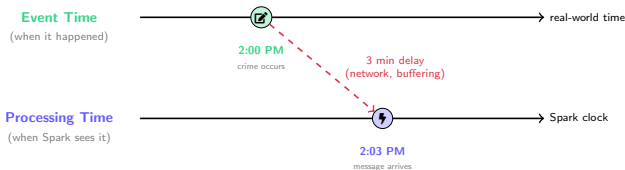
Operation	OK?	Notes
select, filter, map	✓	Stateless — always works
groupBy().count()	✓	Stateful — uses complete/update mode
Join (stream + static table)	✓	Static table must fit in memory
Join (stream + stream)	✓	Must define watermark
orderBy / sort	✗	Cannot sort an infinite table
distinct	✗	Would require infinite memory
limit(n)	✗	Not defined on infinite data

Why These Limitations?

Think about it: How do you “sort” a table that never stops growing? You would need to wait until all data arrives — but it **never** all arrives. Operations that need to see **all** data are impossible on infinite streams.

Event Time vs Processing Time — Why It Matters

A crime happens at 2:00 PM. The officer's phone has no signal. The report reaches Kafka at 2:03 PM. Spark processes it at 2:03:05 PM. Which time do you use for your 5-minute window?



If You Use Processing Time

The crime at 2:00 lands in whatever window is active at 2:03.

Results depend on **network speed**, not reality. **Wrong.**

If You Use Event Time

The crime is **always** in the 2:00–2:05 window, regardless of when it arrives. Results reflect **reality. Correct.**

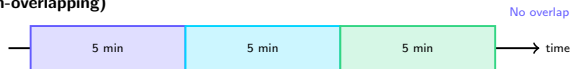
Structured Streaming Default

Spark's `window()` function groups by the timestamp **in your data** (event time), not by when Spark processes it. This is why your JSON must include a timestamp field.

Time Windows — Tumbling vs Sliding

Your boss asks: “How many crimes happened in the last 5 minutes?” You cannot *groupBy* the entire infinite stream — that is unbounded. So you group by **time windows**: fixed slices of time. But which kind of slices?

Tumbling Window (non-overlapping)



Sliding Window (overlapping)



Tumbling

Each event belongs to **exactly one** window.
Use for: fixed reporting intervals.

Sliding

Events can belong to **multiple** windows.
Use for: moving averages, trend detection.

Window Aggregations in Code

```
from pyspark.sql.functions import window, col, count

# TUMBLING window: count crimes per 5-minute window
tumbling = crimes_df \
    .groupBy(
        window("event_time", "5 minutes"),
        "type"
    ).count()

# SLIDING window: 10-min window, sliding every 2 minutes
sliding = crimes_df \
    .groupBy(
        window("event_time", "10 minutes", "2 minutes"),
        "type"
    ).count()
```

Important: You Need a Timestamp Column!

The `window()` function requires an event-time column (e.g., `event_time`). If your JSON has a timestamp, parse it. Otherwise, use the Kafka timestamp column.



Live Cluster Result: 5-Minute Tumbling Windows

All 30 events fell within a single 5-minute window (19:25–19:30 on April 18, 2026). In a real deployment, events would span multiple windows.

window	type	count
{2026-04-18 19:25:00, 2026-04-18 19:30:00}	ASSAULT	7
{2026-04-18 19:25:00, 2026-04-18 19:30:00}	BURGLARY	7
{2026-04-18 19:25:00, 2026-04-18 19:30:00}	THEFT	5
{2026-04-18 19:25:00, 2026-04-18 19:30:00}	CRIMINAL_DAMAGE	4
{2026-04-18 19:25:00, 2026-04-18 19:30:00}	BATTERY	2
{2026-04-18 19:25:00, 2026-04-18 19:30:00}	ROBBERY	2
{2026-04-18 19:25:00, 2026-04-18 19:30:00}	MOTOR_VEHICLE_THEFT	2
{2026-04-18 19:25:00, 2026-04-18 19:30:00}	NARCOTICS	1

Reading the Window Column

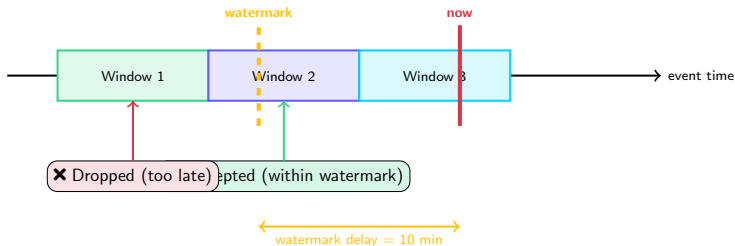
`{start, end}` defines the window boundaries.

What Would Change in Production

With events spread over hours, you would see **multiple windows** — each with its own counts.

Watermarks — Handling Late Data

*Without watermarks, Spark must keep the state for **every window it has ever seen** — because a late event could update any of them. After 24 hours: 288 five-minute windows in memory. After a week: 2,016. Eventually, your executor runs out of memory and crashes.*



Without Watermark

Spark keeps **all** window state forever.
Memory grows **unbounded**!

With Watermark

Spark drops old state once the watermark passes. Memory is **bounded**.

Watermarks in Code

```
# Accept events up to 10 minutes late  
# Drop state for windows older than the watermark  
crimes_with_watermark = crimes_df \  
    .withWatermark("event_time", "10 minutes") \  
    .groupBy(  
        window("event_time", "5 minutes"),  
        "type"  
    ).count()
```

💡 When to Use Watermarks

- Always use watermarks with **window aggregations** in production
- The delay value depends on your data: IoT sensors → seconds; mobile apps → minutes; batch uploads → hours
- For this lab: 10 minutes is a reasonable default

Remember from Week 8

This is similar to `.cache()` controlling memory usage. Watermarks control **how long Spark remembers** partial results.

Outline

- 1 The Streaming Mental Model
- 2 Reading Streams
- 3 Transformations on Streams
- 4 Writing Stream Results**
- 5 End-to-End Pipeline
- 6 Summary

writeStream — The Streaming Output

```
query = type_counts.writeStream \  
  .outputMode("complete") \  
  .format("console") \  
  .trigger(processingTime="5 seconds") \  
  .start()  
  
query.awaitTermination()    # block until query is stopped
```



The Streaming Pipeline

`readStream (source)` → **transformations** (filter, groupBy, window) → `writeStream (sink)`. This is the complete pattern for every streaming job.

Output Modes Explained

Same input, three different outputs. Which rows does Spark write to the sink after each micro-batch?

+ Append

Only **new** rows are written.

*No aggregations
(or after watermark)*

↻ Update

Only **changed** rows are written.

*Best for dashboards
and KV stores*

🗒 Complete

Entire result table re-written each time.

*Only for small results
(few groups)*

Worked Example: `groupBy("type").count()`

Mode	Batch 1 output (input: THEFT, THEFT, BATTERY)	Batch 2 input (+1 THEFT)	Batch 2 output
Append	<i>not allowed with agg</i>	—	—
Update	THEFT:2, BATTERY:1	+THEFT	THEFT:3 (only changed)
Complete	THEFT:2, BATTERY:1	+THEFT	THEFT:3, BATTERY:1 (full table)

Common Mistake

Using complete mode with a high-cardinality groupBy (e.g., grouping by user ID). This re-writes **millions of rows** every micro-batch. Use update mode instead.

Output Sinks and Trigger Options

Output Sinks

Sink	Best For
console	Debugging
memory	Interactive SQL queries
kafka	Chaining pipelines
parquet	Archiving to HDFS
foreach	Custom logic (DB, HTTP)

Trigger Options

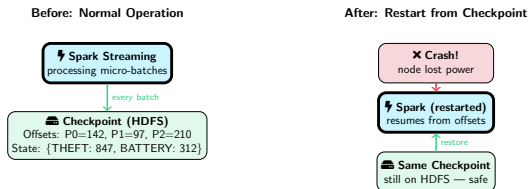
Trigger	Behavior
"5 seconds"	Every 5 seconds
"0 seconds"	ASAP (lowest latency)
once=True	Run once, then stop
availableNow	Process all pending, stop

The Memory Sink — Great for Exploration

Write to an in-memory table, then query it with `spark.sql("SELECT * FROM crimes_live")`. Perfect for notebooks and debugging — **not for production**.

Checkpointing — What Happens When Spark Crashes?

Your streaming job has been running for 6 hours. The executor node loses power. When it restarts, does it re-process 6 hours of data? Or does it lose everything?



What the Checkpoint Contains

- Kafka offsets per partition
- Aggregation state (partial counts)
- Query metadata (schema, config)

```
query = df.writeStream \
    .option("checkpointLocation",
            "hdfs:///checkpoints/crimes") \
    .outputMode("complete") \
    .format("console") \
    .start()
```

The Exactly-Once Guarantee

Kafka **retains** messages (Week 10). Spark **checkpoints** offsets. On crash: Spark re-reads from the checkpointed offset and recomputes the partial batch. No data is lost, no data is double-counted — **exactly-once end-to-end**.

Outline

- 1 The Streaming Mental Model
- 2 Reading Streams
- 3 Transformations on Streams
- 4 Writing Stream Results
- 5 End-to-End Pipeline**
- 6 Summary

The Full Pipeline — Putting It All Together



Lab Workflow

- 1 **Terminal 1:** Run `crime_producer.py` (sends events to Kafka every second)
- 2 **Terminal 2:** Run Spark streaming job (reads from Kafka, aggregates, prints)
- 3 **Watch:** Counts update every 5 seconds as new events arrive

Monitoring — Query Progress

```
query = type_counts.writeStream \  
    .outputMode("complete") \  
    .format("console") \  
    .trigger(processingTime="5 seconds") \  
    .start()  
  
# Check query status  
print(query.status)           # is it running? waiting for data?  
print(query.lastProgress)     # timing, rows processed, state info  
print(query.recentProgress)   # list of recent micro-batches
```

Spark UI: Streaming Tab

- Input rate (rows/sec)
- Processing rate (rows/sec)
- Batch duration (ms)

Key Metric

If **processing rate** < **input rate**,
your pipeline is falling behind!
Fix: more partitions or faster logic.

Recall from Week 8

The Spark Web UI at <http://master:4040> now has a **Structured Streaming** tab alongside the Jobs/Stages/Storage tabs you already know.

Outline

- 1 The Streaming Mental Model
- 2 Reading Streams
- 3 Transformations on Streams
- 4 Writing Stream Results
- 5 End-to-End Pipeline
- 6 Summary

Key Takeaways

- 1 **Structured Streaming** = “infinite table” processed with the **same DataFrame API** you already know from Week 7–8
- 2 `readStream` + `writeStream` replace `read` + `write`. Everything in between is identical.
- 3 Three output modes: **append** (new rows), **update** (changed), **complete** (full rewrite)
- 4 **Time windows** (tumbling/sliding) enable real-time aggregation over fixed intervals
- 5 **Watermarks** bound late data and prevent unbounded memory growth
- 6 **Kafka** → **Spark Streaming** is the standard real-time pipeline
- 7 **Checkpointing** saves offsets + state to HDFS, enabling crash recovery and **exactly-once** end-to-end guarantees
- 8 **Event time** (not processing time) determines which window a record belongs to — Spark handles late data via **watermarks**



The Week 10–11 Stack

Kafka (transport) + **Spark Structured Streaming** (processing)

Spark Streaming vs Flink vs Kafka Streams

Spark Structured Streaming is not the only stream processing engine. How does it compare?

Feature	Spark Str. Streaming	Apache Flink	Kafka Streams
Processing model	Micro-batch (default)	True event-by-event	Event-by-event
Latency	100ms – seconds	Low milliseconds	Low milliseconds
State management	Built-in (RocksDB)	Built-in (RocksDB)	Built-in (RocksDB)
Deployment	YARN / K8s cluster	Standalone / YARN / K8s	Library (any JVM app)
API style	SQL / DataFrame	DataStream + SQL	Java DSL
Best for	Unified batch + stream	Low-latency, complex CEP	Lightweight, inside app

Why We Use Spark Structured Streaming in This Course

You already know the DataFrame API from Weeks 7–8. Structured Streaming reuses that **same API**, meaning you learn one framework for both batch and streaming. In industry, the choice depends on latency requirements: if sub-100ms latency is critical, consider Flink.

A Common Misconception

Kafka Streams is **not** the same as Apache Kafka. Kafka is the message bus (Week 10). Kafka Streams is a separate stream-processing library that happens to read from Kafka — just like Spark Structured Streaming does.

What's Next

Week 12: Midterm 2 + Advanced Topics Review

- Midterm 2 covers Weeks 7–11 (Spark, MLlib, Kafka, Streaming)
- Review all pipeline patterns: batch vs real-time
- Milestone M4 (Spark) due end of Week 11

Today's Lab: Spark Streaming with Kafka

- Write a Python producer that simulates live crime events
- Create a Spark Structured Streaming job reading from Kafka
- Compute real-time crime counts per type (5-min tumbling windows)
- Compute running arrest rate per district
- Observe micro-batch execution in the Spark UI

Milestone M5 Reminder

Due Week 12: End-to-end streaming pipeline (Kafka → Spark Streaming → output). This lab gives you the building blocks.